O(1) Bit Hacks and Queries

(Note: In all these examples, bits are 0-indexed from right to left. The rightmost bit is the 0-th bit).

1. The Mask: (1 << i)

Before manipulating a specific bit, you need a "mask". Shifting the integer 1 left by i positions creates a number where only the i-th bit is 1, and everything else is 0.

1 << 3 becomes 00001000.

2. Checking the i-th bit

`Formula: (x & (1 << i)) != 0`

You want to know if the i-th bit of x is 1. By using the bitwise *AND* with your mask, you destroy all other bits (because anything *AND* 0 is 0).

If the i-th bit in x was 0, the whole result becomes 0.
If the i-th bit in x was 1, the result will be greater than 0 (specifically, it will equal 2^i).

3. Setting the i-th bit

`Formula: x = x | (1 << i)`

You want to force the i-th bit to be 1, regardless of what it currently is.
Using the OR operator with the mask guarantees the i-th bit becomes 1, while leaving all other bits completely unchanged (because anything OR 0 remains itself).

4. Clearing the i-th bit

`Formula: x = x & ~(1 << i)`

You want to force the i-th bit to be 0.

1. Create the mask: 00001000
2. Invert the mask using *NOT* (~): 11110111
3. *AND* it with x. The 0 in the i-th position acts like a black hole, destroying whatever bit was there in x. The 1s act like a transparent window, letting all other bits pass through unchanged.

5. Toggling the i-th bit

`Formula: x = x ^ (1 << i)`

You want to flip the i-th bit (1 to 0 or 0 to 1). XOR is perfect for this. XORing any bit with 1 flips it. XORing any bit with 0 leaves it alone.

6. Isolating the Lowest Set Bit

`Formula: x & -x`

This is arguably the most beautiful bit hack in computer science. It finds the rightmost 1 in the binary representation of x and zeros out everything else.

- Let x = 10 (which is 00001010 in binary).
- The lowest set bit is at position 1 (value of 2).
- From above, we know Two's Complement makes -x equal to ~x + 1.
- ~x is 11110101. Adding 1 gives -x as 11110110.
- (00001010) & (11110110) = 00000010 (which is 2).

This O(1) trick is the entire mathematical foundation of the Binary Indexed Tree (Fenwick Tree), a crucial data structure for efficient range queries.

7. Checking if a Number is a Power of 2

`Formula: (x & (x - 1)) == 0 (Assuming x > 0)`

Powers of 2 (like 2, 4, 8, 16) only have exactly one bit set to 1 in their binary representation.
Subtracting 1 from a number borrows from the lowest set bit.

It flips that lowest 1 to a 0, and turns all the 0s to its right into 1s.

- Let x = 16 (binary 10000).
- x - 1 = 15 (binary 01111).
- 10000 & 01111 = 00000.

If x is not a power of 2, the *AND* operation will leave some higher set bits untouched, resulting in a non-zero value.

8. Popcount (Counting Set Bits)

You often need to know exactly how many 1s are in a number (the "Hamming Weight"). While you could write a while loop using x = x & (x - 1) to strip away the lowest set bit until the number is 0, you shouldn't.

Modern CPUs have a dedicated hardware instruction for this (POPCNT).

In C++: Use the compiler intrinsic __builtin_popcount(x) for 32-bit integers, and __builtin_popcountll(x) for 64-bit integers.

> When adding 1 to a binary number, flip all trailing 1s to 0s until you encounter the first 0, flip that 0 → 1, and keep the remaining bits unchanged.
>
> What Happens When You Subtract 1 in Binary: the rightmost 1 becomes 0, and all bits to its right become 1.

Brian Kernighan Trick

Key idea:

n & (n - 1) removes the lowest set bit.

n = 101100
n - 1 = 101011

So if we repeatedly apply it, we remove one set bit each time.

Algorithm:

```
count = 0
while(n > 0){
    n = n & (n - 1)
    count++
}
```

The number of iterations = number of set bits.